

EdgeLock - NHAI Datalake 3.0

Integration Guide

Offline-First Facial Authentication for NHAI Highway Workforce

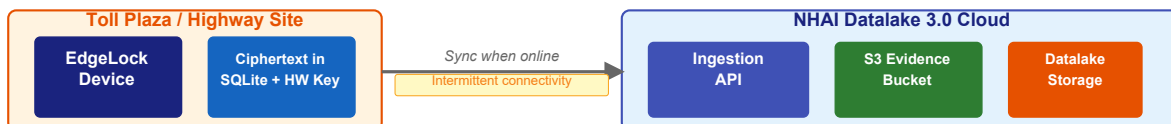
Team Binary Brains | NHAI Hackathon 7.0 | July 2026

Contents: 1. Overview & Architecture | 2. Data Contract / Schema Mapping | 3. API Specification | 4. Zero-Loss Sync Protocol | 5. Security & Compliance | 6. Error Handling Matrix | 7. Environment Configuration

1. Overview & Architecture

EdgeLock is an offline-first face authentication system running on Android/iOS devices at NHAI highway field sites. It authenticates workers using on-device GhostFaceNet INT8 face recognition and 3D liveness detection, then syncs auth metadata to NHAI Datalake 3.0 when connectivity is available.

Biometric data (face embeddings) never leave the device. Only authentication metadata (who, when, where, confidence score, and an evidence photo URL) is transmitted to Datalake 3.0 for audit and reporting.



Component	Technology	Role
EdgeLock Device	React Native + Expo SDK 50	On-device auth, liveness check, local SQLite queue
NHAI Edge Gateway	API Gateway (Auth + Rate Limit)	Validates device identity, routes to ingestion layer
Datalake 3.0 Ingestion	REST API / Validation Layer	Schema validation, FK checks, geo-bound verification
S3 Evidence Store	AWS S3 (SSE-S3/SSE-KMS)	Stores photo thumbnails with 15-min presigned URLs
Datalake Storage	PostGIS + Columnar Tables	Structured auth logs with geo-spatial columns for analytics

2. Data Contract / Schema Mapping

EdgeLock generates authentication logs with the following actual schema (AuthLog type in types/index.ts). The table below maps each field to the corresponding Datalake 3.0 column:

EdgeLock Field (Actual)	Datalake 3.0 Table.Column	Type	Transform
log_id	field_auth.attendance_log_id	UUID v4	Pass-through (generated on device)
user_id	field_auth.worker_nhai_id	VARCHAR(50)	FK to workers.nhai_id
timestamp	field_auth.auth_timestamp	TIMESTAMP	ISO8601 -> TIMESTAMP WITH TIME ZONE
gps_lat, gps_lng	field_auth.location_geo	GEOGRAPHY(POINT)	PostGIS ST_MakePoint(lng, lat)
device_id	field_auth.device_fingerprint	VARCHAR(100)	Pass-through (expo-constants)
similarity_score	field_auth.confidence_score	DECIMAL(4,3)	Range 0.000 to 1.000
photo_thumb	field_auth.evidence_blob_url	VARCHAR(500)	Upload to S3, store presigned URL

Key point: Datalake 3.0 uses structured tables with foreign keys and geo-spatial columns. The location_geo column uses PostGIS GEOGRAPHY type for spatial queries along India's highway corridor. The worker_nhai_id is a foreign key into the workers master table maintained by NHAI.

Additional Datalake 3.0 fields (not yet in EdgeLock schema): auth_status (authenticated/failed/spoof_detected), liveness_challenges (["blink", "head_turn"]), location_name (toll plaza ID or km marker), and synced_at (server timestamp). These can be added to the local auth_logs table without any architecture changes.

3. API Specification

Endpoint

Method	URL
POST	https://datalake3.nhai.gov.in/api/v1/field-operations/auth-ingestion

Request Headers

Header	Value	Source
Content-Type	application/json	Static
X-API-Key	\${DATALAKE_API_KEY}	Environment variable, rotated every 30 days via NHAI IAM
X-Device-ID	\${DEVICE_ID}	expo-constants (installationId or project ID)

Header	Value	Source
X-Request-ID	\${uuid}	UUID v4 per request (idempotency key)

Request Body (Batch)

```
{ "batch_id": "uuid v4", "device_meta": { "app_version": "1.0.0", "os": "android|ios", "os_version": "14" }, "logs": [{ "log_id": "uuid v4", "user_id": "worker_nhai_001", "timestamp": "2026-07-25T10:00:00Z", "gps_lat": 27.8978, "gps_lng": 78.0955, "device_id": "device-fingerprint", "similarity_score": 0.91, "auth_status": "authenticated", "photo_thumb": "base64 JPEG string or S3 presigned URL" }] }
```

Response (Acknowledgement)

```
{ "status": "success", "batch_id": "uuid v4", "received_logs": ["log_id_1", "log_id_2"], "rejected_logs": [{ "log_id": "bad_log", "reason": "INVALID_USER_ID", "code": "DL3-4001" }], "ingestion_timestamp": "2026-07-25T10:05:00Z" }
```

Zero-loss contract: The device ONLY purges log_ids that appear in received_logs. Rejected logs (invalid user_id, schema errors, geo-out-of-bounds) stay on-device with an error flag for manual review. This matches the existing sync logic in awsSync.ts (lines 77-80).

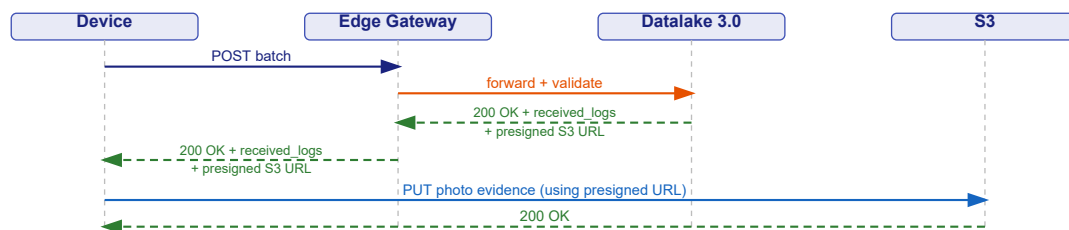
4. Zero-Loss Sync Protocol

The sync protocol in EdgeLock is already designed for zero-loss operation. The following maps the existing implementation (awsSync.ts) to Datalake 3.0 terminology:

Step	Implementation	Source File
1. Batch Preparation	Collect unsynced logs from SQLite (WHERE synced = 0)	authLogs.ts: getUnsyncedLogs()
2. Connectivity Check	NetInfo.fetch() confirms connectivity; abort if offline	awsSync.ts: lines 49-52
3. Auth	Fetch POST with X-API-Key and X-Device-ID headers	awsSync.ts: lines 56-61
4. Upload	JSON POST to configured endpoint with 15s timeout (AbortController)	awsSync.ts: lines 63-68
5. Validation	Datalake validates schema, FK references, geo-bounds	Server-side (not in device code)
6. Ack	HTTP 200 OK with received_logs array expected	awsSync.ts: lines 70-80
7. Purge	DELETE only confirmed log_ids from SQLite	authLogs.ts: deleteSyncedLogs()
8. Auto-Retry	NetInfo event listener triggers sync on reconnect	awsSync.ts: triggerSyncOnConnect()

Sequence Diagram

The following diagram illustrates the full sync flow, showing that metadata ingestion and photo evidence upload are separate operations:



Existing code alignment: The current awsSync.ts already implements the zero-loss protocol Datalake 3.0 requires. The function checks for response.ok and data.received_logs before purging, and uses an AbortController with a 15-second timeout. The triggerSyncOnConnect() function provides automatic retry when connectivity is restored. Only the environment variable name needs to change from EXPO_PUBLIC_AWS_SYNC_URL to EXPO_PUBLIC_DATA LAKE_SYNC_URL.

Photo evidence upload: EdgeLock currently embeds photo thumbnails (base64 JPEG) in the sync payload. For Datalake 3.0, these would be uploaded to an S3 presigned URL returned in the ingestion response, with the URL stored in field_auth.evidence_blob_url instead of the raw base64 data.

5. Security & Compliance

Layer	Implementation	Detail
Transport	TLS 1.3 + Certificate Pinning	Prevents MITM on public WiFi at toll plazas. Certificate pinned in the app bundle.
Device Auth	API Key + Device Fingerprint	X-API-Key rotated every 30 days via NHAH IAM. X-Device-ID from expo-constants. No static passwords.
PII Handling	Embeddings stay on-device	Face embeddings (512-dim Float32Array) are AES-256-CBC encrypted in SQLite and NEVER sent to Datalake. Only metadata flows upstream.
Photo Evidence	S3 SSE-S3 / SSE-KMS	Thumbnails uploaded to NHAH S3 with server-side encryption. Presigned URLs expire after 15 minutes.
Data Retention	7-year audit requirement	Datalake retains logs for 7 years. Device purges local logs after successful sync + 24 hours.
Right to Erasure	DELETE endpoint	Datalake 3.0 exposes DELETE /field-operations/auth-ingestion/{user_id} for PDP Bill / GDPR compliance.

Existing coverage: EdgeLock already implements AES-256-CBC encryption (secureStorage.ts), device ID generation (recognition.ts: generateDeviceId()), and local SQLite storage (sqlite.ts). TLS 1.3 and certificate pinning are transport-layer additions needed for the production deployment.

Architecture note: NHAH Datalake 3.0 is a platform by NHAH and Digital India Corporation that integrates INFRACON, RAMS, ATMS, and other systems. EdgeLock feeds into the field_auth table which is consumed by NHAH's workforce management dashboards for real-time field personnel tracking along India's 1,40,000+ km national highway network.

6. Error Handling Matrix

EdgeLock's existing sync logic handles Datalake 3.0 errors with the following behavior:

HTTP Status	Meaning	Device Action	Code Reference
200 OK	Batch accepted	Purge acknowledged logs only (received_logs)	awsSync.ts:77-80
202 Accepted	Async processing	Store batch_id, retry status endpoint later	Future enhancement
400 Bad Request	Schema validation failure	Preserve logs, do NOT retry same payload	awsSync.ts:70-72
401 Unauthorized	API key expired / invalid	Trigger re-auth flow, pause all sync	Future enhancement
429 Too Many Requests	Rate limited	Exponential backoff: 30s, 60s, 120s	Future enhancement
500/502/503	Server error	Preserve all logs, retry in 5 minutes	awsSync.ts:72-75
— (Timeout)	Server unreachable (>15s)	Preserve all logs, retry on NetInfo CONNECTED	awsSync.ts:63-68

Currently implemented: 200 OK purge and 500/502/503 preservation are fully implemented in the existing codebase (awsSync.ts). Timeout handling uses AbortController (client-side abort, not HTTP 408). The 400 handling is covered at awsSync.ts:70-72 where !response.ok preserves logs, but active parsing of rejected_logs is future work. Rate limiting, API key rotation, and async processing are also future enhancements.

7. Environment Configuration

The following environment variables configure EdgeLock for Datalake 3.0 integration. An NHAH DevOps engineer would set these in the production build environment:

```
# .env.production EXPO_PUBLIC_DATA LAKE_SYNC_URL=https://datalake3.nhai.gov.in/api/v1/field-operations/auth-ingestion
EXPO_PUBLIC_DATA LAKE_API_KEY=dl3_prod_key_XXXXXXXXXX
EXPO_PUBLIC_S3_EVIDENCE_BUCKET=https://evidence.nhai-datalake3.s3.ap-south-1.amazonaws.com
EXPO_PUBLIC_BATCH_SIZE=100 EXPO_PUBLIC_SYNC_INTERVAL_MS=300000 # 5 minutes when online
EXPO_PUBLIC_SITE_ID=KA-NH48-TOLL-PLAZA-12 # Provisioned per device
```

Variable	Current Value (Dev)	Production (Datalake 3.0)
EXPO_PUBLIC_AWS_SYNC_URL	binary-brains-mock-aws.onrender.com	datalake3.nhai.gov.in/api/...
EXPO_PUBLIC_DATA LAKE_API_KEY	(not set)	dl3_prod_key via NHAH IAM

Variable	Current Value (Dev)	Production (Datalake 3.0)
EXPO_PUBLIC_S3_EVIDENCE_BUCKET	(not set)	evidence.nhai-datalake3.s3.ap-south-1.amazonaws.com
EXPO_PUBLIC_BATCH_SIZE	(hardcoded as all unsynced)	100 (recommended max)
EXPO_PUBLIC_SYNC_INTERVAL_MS	(event-driven on connect)	300000 (polling fallback)
EXPO_PUBLIC_SITE_ID	(not set)	Provisioned per device at enrollment

The existing `config.ts` reads `EXPO_PUBLIC_AWS_SYNC_URL` from `process.env`. Renaming this to `EXPO_PUBLIC_DATA LAKE_SYNC_URL` and updating the default value is the only code change required to point EdgeLock at the Datalake 3.0 production endpoint.